Modern Information Retrieval

Final Project Specification

Dual-Engine Document Search & RAG System

Proposed & Designed by:

Saba Famil Feizi

Project Objective:
Design, implement, and thoroughly evaluate a modern search engine that bridges classical lexical retrieval methodologies with cutting-edge semantic vector search and LLM-powered answer generation. Students will build a robust, full-stack system featuring synchronized indexes, multiple search algorithms, and an intuitive user interface tailored for document discovery.

1 Introduction and Background

Information Retrieval (IR) has undergone a paradigm shift over the last decade. Classical lexical search methods, which rely on exact keyword matching (such as the Vector Space Model and Probabilistic Models like BM25), form the foundation of historical search engines. While highly efficient and transparent, they often struggle with the "vocabulary mismatch" problem—where users query with terms different from those used in the documents.

Modern IR systems address this by incorporating **Semantic Search** via deep learning. By converting text into dense vectors (embeddings), these systems can retrieve documents based on conceptual meaning rather than exact word overlap. Furthermore, **Retrieval-Augmented Generation (RAG)** pipelines take these retrieved documents and pass them to Large Language Models (LLMs) to synthesize natural language answers, fundamentally changing how users interact with information.

This final project challenges you to build a system that implements *both* paradigms. You will construct a unified architecture capable of switching seamlessly between traditional term-based retrieval and modern semantic generation, updating a dual-index system in real-time as the corpus changes.

2 System Architecture & Data Flow

The core of this project requires building a search system that maintains two parallel data stores. The system must handle two primary pipelines: **Indexing** and **Querying**.

2.1 The Indexing Pipeline

When an administrator uploads documents (e.g., PDF, Word) through the dashboard, the system must perform the following steps:

1. **Parsing & Extraction:** Raw text and metadata are extracted from the file formats.
2. **Chunking:** Because LLMs and Embedding models have context window limits, the text must be split into logical, overlapping chunks (e.g., 500 words per chunk with a 50-word overlap).
3. **Dual-Indexing:**
    - *Inverted Index:* The chunked text is tokenized, stemmed/lemmatized, and added to an inverted index. Postings lists are updated with term frequencies.
    - *Vector Database:* Each chunk is passed through an embedding model to generate a dense vector representation, which is then stored in a Vector DB alongside its metadata (document ID, chunk text).

2.2 The Querying Pipeline

When a user submits a query via the frontend, the system routes the request based on the selected search methodology:

- **Lexical Route:** The query is preprocessed and matched against the Inverted Index using either VSM or BM25 scoring.
- **Semantic/RAG Route:** The query is embedded using the same model used during indexing. The Vector DB performs a similarity search (e.g., Cosine Similarity) to retrieve the top-k chunks. These chunks are then inserted into a prompt template and sent to an LLM to generate a final answer.

3 Detailed Component Specifications

3.1 1. Document Processing & Synchronization

Handling real-world documents is a critical skill in modern IR.

- **Supported Formats:** The system must natively parse `.pdf` and `.docx` files. You may use external libraries (e.g., PyMuPDF, python-docx) for parsing.
- **Dynamic Synchronization:** The Inverted Index and Vector Database must remain perfectly synchronized. If an administrator deletes a document from the corpus via the UI, the system must immediately remove all associated postings from the inverted index and all associated vectors from the vector database.

3.2 2. Retrieval Engine Methodologies

You must implement three distinct search pipelines. You are encouraged to build the inverted index from scratch to demonstrate your understanding of underlying data structures.

3.2.1 Method A: Vector Space Model (VSM) & Optimizations

- **Scoring:** Implement standard TF-IDF weighting for queries and documents.
- **Inexact Top-K Search:** Standard VSM scoring over a large corpus is computationally expensive. You must implement at least one inexact retrieval technique to speed up the process. Examples include *Champion Lists*, *Tiered Indexes*, or *Index Elimination* (only scoring documents that contain high-IDF query terms).
- **Pseudo Relevance Feedback (PRF):** Implement Rocchio's algorithm or a similar PRF method. When toggled on, the system should perform an initial retrieval, assume the top k documents are relevant, expand the query with the most frequent/important terms from these documents, and execute a second retrieval to improve recall.

3.2.2 Method B: BM25 Probabilistic Model

- Implement the Okapi BM25 ranking function. Ensure parameters k_1 (term frequency saturation) and b (length normalization) are tunable or set to standard empirical baselines (e.g., $k_1 = 1.5, b = 0.75$).

3.2.3 Method C: Retrieval-Augmented Generation (RAG)

- **Retrieval:** Use the user's query to perform a dense vector similarity search against the Vector DB.
- **Generation:** Pass the top retrieved chunks into an LLM context window.
- **Citation Requirement:** The generated response *must* cite the retrieved documents used in generation. The prompt to the LLM should instruct it to output citations (e.g., "[Doc 1]") corresponding to the retrieved chunks provided in its context.

3.3 3. Full-Stack User Interface

The system must be accessible via a web browser, divided into two primary views:

3.3.1 Admin Dashboard

- **Upload Interface:** A drag-and-drop or file selection area to add new files.
- **Corpus Management:** A table listing all currently indexed documents (Title, File Type, Date Added).
- **Deletion Functionality:** A mechanism to remove documents from the system, which triggers the backend synchronization logic to clean the indexes.

3.3.2 Search Frontend

- **Query Input:** A prominent search bar.
- **Algorithm Selector:** Radio buttons or a dropdown allowing the user to select the retrieval method (VSM, BM25, or RAG).
- **PRF Toggle:** A checkbox to enable/disable Pseudo Relevance Feedback when VSM is selected.
- **Results Display:**
  - For VSM and BM25: Display a ranked list of relevant documents with titles, scores, and text snippets highlighting the matched query terms.
  - For RAG: Display the synthesized LLM answer prominently at the top, complete with citation markers. Below the answer, display the source document chunks that were cited.

4 Recommended Technology Stack

To ensure the project remains accessible and can run on standard student hardware without incurring cloud costs, the following open-source/free-tier stack is strongly recommended:

System Component	Recommended Tools & Libraries
Document Parsing	`PyMuPDF` (fitz) or `pdfplumber` for PDFs; `python-docx` for Word files.
Text Chunking	`LangChain` Document Loaders and RecursiveCharacterTextSplitter.
Vector Database	`ChromaDB` (runs entirely local/in-memory) or `FAISS`.
Embedding Model	HuggingFace `sentence-transformers` (e.g., `all-MiniLM-L6-v2`) running locally on CPU.
Generative LLM	Google Gemini API (Generous free tier), GroqCloud API, or `Ollama` (for fully local execution of Llama-3/Phi-3).
Web Frontend	HTML/Tailwind CSS/JS for custom builds, or Python UI frameworks like `Streamlit` or `Gradio` for rapid prototyping.
Backend Framework	`FastAPI` or `Flask` to connect the UI to the IR Python logic.

5 Grading Rubric (100 Points Total)

The project will be evaluated based on functionality, algorithmic accuracy, system design, and the quality of the user interface.

rubrichead **Category**	**Criteria**	**Points**
Data Processing & Indexing	Accurate parsing of PDF/Word files. Logical chunking implementation. Successful creation of the classical Inverted Index and Vector Database.	**20**
Index Synchronization	The system correctly and efficiently updates BOTH indexes when documents are added or removed from the admin dashboard.	**15**
Classical Retrieval	Correct implementation of VSM (TF-IDF), BM25, and at least one Inexact Top-K optimization.	**20**
Query Expansion	Correct implementation of Pseudo Relevance Feedback (PRF) that visibly alters/improves search results.	**10**
Semantic RAG Pipeline	Successful embedding of queries, semantic retrieval from the Vector DB, and accurate LLM generation with proper document citations.	**20**
UI & System Integration	A functional, bug-free web frontend encompassing both the Admin Dashboard (upload/delete) and the User Search view.	**15**

6 Bonus Opportunities (Up to 15 Extra Points)

Students who demonstrate exceptional ambition can earn bonus points by implementing the following features:

- ⋆ **Web Scraping Module (+5 pts):** Add functionality to the Admin Dashboard allowing the user to input a URL. The system should scrape the web page, extract the main text, and add it to the corpus seamlessly.
- ⋆ **Innovative Results Visualization (+5 pts):** Go beyond a simple list of links. Implement keyword heatmaps, interactive document relation graphs, or advanced highlighting of retrieved chunks.
- ⋆ **Advanced RAG Techniques (+5 pts):** Implement advanced RAG routing, query rewriting before retrieval, or re-ranking models (like Cohere Re-rank or Cross-Encoders) to refine the top-k results before passing them to the LLM.

7 Deliverables & Submission Guidelines

Your final submission must include the following components:

1. **Source Code Repository:** Provide a link to your Git repository (GitHub/GitLab). Ensure your code is clean, modular, and adequately commented.
2. **README.md:** A detailed guide including:
   - System architecture overview.

- Step-by-step setup and installation instructions (e.g., `pip install -r requirements.txt`).
- Instructions on how to set up necessary API keys for the LLM.

3. **Video Demonstration:** A 5-7 minute video demonstrating the system's capabilities. You must show:
   - Uploading a new document and querying it.
   - Deleting a document and proving it no longer appears in search results.
   - Executing identical queries across VSM, BM25, and RAG to compare the results and UI presentation.

Good luck! This project is designed to give you hands-on experience with the systems powering modern enterprise search and AI assistants.